Parallel Computation for the All-Pairs Suffix-Prefix Problem

Felipe A. Louza[1(✉)], Simon Gog[2], Leandro Zanotto[1], Guido Araujo[1], and Guilherme P. Telles[1]

[1] Institute of Computing, University of Campinas, São Paulo, Brazil
{louza,guido,gpt}@ic.unicamp.br, leandro.zanotto@reitoria.unicamp.br
[2] Institute of Theoretical Informatics, Karlsruhe Institute of Technology, Karlsruhe, Germany
gog@kit.edu

Abstract. We show how to parallelize the optimal algorithm proposed by Tustumi *et al.* [19] to solve the all-pairs suffix-prefix matching problem for general alphabets. We compared our parallel algorithm with SOF [17], a practical solution for DNA sequences that exhibits good time and space performance in multithreading environments. The experimental results showed that our parallel algorithm achieves a consistent speedup when compared with the sequential algorithm, and it is competitive with SOF when the minimum overlap length is small.

Keywords: Suffix-prefix matching · Parallel algorithm · Multithreading · Suffix array · LCP array

1 Introduction

Given a collection of strings the all-pairs suffix-prefix matching problem (APSP) is to find all longest overlaps among string ends [7]. This problem is well know in stringology [14] and appears often as a bottleneck part of DNA assembly, where the number of strings ranges from thousands to billions [3]. Other applications include EST clustering [9] and approximating the shortest common superstring [7].

The APSP has been approached in different ways. In practice with DNA sequences, filtering strategies have been used and rendered very efficient algorithms that are able to cope with the huge scale of DNA assembly [5,18]. On the theoretical side, optimal algorithms exist [8,15,19] that are able to handle strings from general alphabets, and, while not beating specialized algorithms for DNA, still perform fairly.

In this article we introduce a parallel algorithm for the APSP. Our solution builds on a previous optimal sequential algorithm by Tustumi *et al.* [19]. We were able to obtain a consistent speedup with a small memory footprint with this approach. Experimental results showed that our algorithm is competitive with the practical solution SOF [17] when the minimum overlap length is small.

S. Inenaga et al. (Eds.): SPIRE 2016, LNCS 9954, pp. 122–132, 2016.
DOI: 10.1007/978-3-319-46049-9_12

The next sections are organized as follows. In Sect. 2 we introduce notation and discuss the practical and the optimal algorithms to solve the APSP. In Sect. 3 we present the optimal algorithm by Tustumi *et al.*In Sect. 4 we present our parallel algorithm and in Sect. 5 we show experimental results. In Sect. 6 we conclude the article.

2 Preliminaries

2.1 Notation

Let S be a string of length n over an ordered alphabet Σ. The i-th symbol of S is denoted by $S[i]$ and the substring including symbols in the interval $[i, j]$, $1 \leq i \leq j \leq n$, is denoted by $S[i, j]$. A prefix of S is a substring of the form $S[1, i]$ and a suffix is a substring of the form $S[i, n]$, which will be denoted by S_i. We use the symbol $<$ for the lexicographic order relation between strings.

The suffix array of $S[1, n]$, SA, is an array of integers in the range $[1, n]$ that gives the lexicographic order of all suffixes of S, such that $S_{\mathsf{SA}[1]} < S_{\mathsf{SA}[2]} < \ldots < S_{\mathsf{SA}[n]}$ [6,13]. We denote the position of suffix S_i in SA as $\mathsf{pos}(S_i)$. The LCP-array is an array of integers that stores the length of the longest common prefix (lcp) of two consecutive suffixes in SA, such that $\mathsf{LCP}[1] = 0$ and $\mathsf{LCP}[i] = lcp(S_{\mathsf{SA}[i]}, S_{\mathsf{SA}[i-1]})$ for $1 < i \leq n$. Both SA and the LCP-array can be constructed in linear time [10,16].

The range minimum query (rmq) with respect to the LCP gives the smallest lcp value in an interval of SA. We define $rmq(i, j) = \min_{i<k\leq j}\{\mathsf{LCP}[k]\}$. Given a string $S[1, n]$ and its LCP-array, it is easy to see that $lcp(S_{\mathsf{SA}[i]}, S_{\mathsf{SA}[j]}) = rmq(i, j)$, with $1 \leq i < j \leq n$.

Let $\mathcal{S} = S^1, S^2, \ldots, S^m$ be a collection of strings of lengths $n_i = |S^i|$, $\forall i \in [1, n]$. The generalized suffix array of $\mathcal{S}$ is the suffix array of the concatenated string $S^{cat} = S^1\$_1 S^2\$_2 \ldots S^m\$_m$ of length $N = m + \Sigma_{i=1}^{m} n_i$, where each symbol $\$_i$ is a distinct separator that does not occur in Σ, precedes every symbol in Σ, and $\$_i < \$_j$ if $i < j$. For a suffix $S^{cat}_{\mathsf{SA}[i]}$, we denote the prefix of $S^{cat}_{\mathsf{SA}[i]}$ that ends at the first separator $\$_j$ by $S^{\$}_{\mathsf{SA}[i]}$. The generalized suffix array can also be constructed in linear time [11].

For a clearer notation, we introduce the arrays STR and SA'. STR indicates which string in $\mathcal{S}$ a suffix came from, that is, $\mathsf{STR}[i] = j$ if the suffix $S^{\$}_{\mathsf{SA}[i]}$ ends with symbol $\$_j$. SA' holds the position of a suffix with respect to the string it came from (up to the separator), defined as $\mathsf{SA}'[i] = k$ if $S^{\$}_{\mathsf{SA}[i]} = S^j_k\$_j$. In other words, STR and SA' specify the order of all suffixes in the collection. We will denote the generalized suffix array enhanced with the arrays STR, SA' and LCP as GESA. The GESA of the collection $\mathcal{S} = \{\mathtt{aac}, \mathtt{aca}, \mathtt{aa}, \mathtt{caa}\}$ is illustrated in Fig. 1.

Let S^k be the j-th (lexicographically) smallest string in $\mathcal{S}$. P is an array of $m + 1$ integers that stores in $\mathsf{P}[j]$ the position of the complete suffix $S^k[1, n_k]$ in GESA. We define $\mathsf{P}[0] = m + 1$. Let the interval $B^j = (\mathsf{P}[j-1], \mathsf{P}[j]]$ be a block of GESA corresponding to S^k. GESA can be partitioned into m blocks

	i	SA	LCP	STR	SA′	$S^{\$}_{\mathsf{SA}[i]}$
	1	4	0	1	4	$\$_1$
	2	8	0	2	4	$\$_2$
	3	11	0	3	3	$\$_3$
	4	15	0	4	4	$\$_4$
P[0] →	5	7	0	2	3	a$\$_2$
	6	10	1	3	2	a$\$_3$
	7	14	1	4	3	a$\$_4$
P[1] →	8	9	1	3	1	aa$\$_3$
	9	13	2	4	2	aa$\$_4$
P[2] →	10	1	2	1	1	aac$\$_1$
	11	2	1	1	2	ac$\$_1$
P[3] →	12	5	2	2	1	aca$\$_2$
	13	3	0	1	3	c$\$_1$
	14	6	1	2	2	ca$\$_2$
P[4] →	15	12	2	4	1	caa$\$_4$

Fig. 1. The GESA of $\mathcal{S} = \{\mathtt{aac}, \mathtt{aca}, \mathtt{aa}, \mathtt{caa}\}$. Suffixes in block 1 are highlighted.

$B^1, B^2, \ldots, B^m$, one for each string S^k in $\mathcal{S}$. In Fig. 1, block $B^1 = (5, 8]$ of the corresponding string $S^3 = \mathtt{aa}$ is shown by a gray rectangle.

2.2 APSP

The all-pairs suffix-prefix matching problem (APSP) is to find, for all pairs of strings S^i and S^j in $\mathcal{S}$, the longest suffix of S^i that is a prefix of S^j [7]. In other words, S^i overlaps S^j. The solution of the APSP can be stored in an "overlap" squared matrix Ov of size m^2, where $\mathsf{Ov}[i, j]$ represents the length of the longest suffix of S^i that overlaps S^j. Ov can also be stored using a compact representation [2].

Practical Algorithms: The most demanding application for the APSP currently is overlap detection for DNA assembly, which has been solved much faster in practice by non-optimal algorithms. SGA [18] and Readjoiner [5] are genome assemblers that have a very fast and isolated overlap detection stage with very low space consumption, which find all suffix-prefix overlaps (not only the longest overlaps). Recently, Rachid and Malluhi [17] presented SOF, a practical algorithm to solve the APSP for DNA sequences that is competitive with the genome assemblers. SGA, Readjoiner and SOF may be executed in multithreading environments. Previous experiments [17] have shown that SOF has a better performance with multiple threads.

Optimal Algorithms: The APSP has been solved in optimal time by Gusfield *et al.* [8] in 1992 through generalized suffix trees [20] and stacks. In 2010, Ohlebusch and Gog [15] improved memory usage through enhanced suffix arrays [1,13] and stacks, reducing the practical running time. Recently,

Tustumi *et al.* [19] proposed a different traversal of the enhanced suffix array and replaced stacks by linked lists to achieve an even better practical running time. We stress that all these algorithms are theoretically optimal and are able to deal with strings from general alphabets.

3 Related Work

The algorithm by Tustumi *et al.* [19] solves the APSP in optimal $O(N + m^2)$ time, based on the following remarks.

All suffixes that are a prefix of S^k are either in positions prior to $\mathsf{pos}(S^k)$ or are identical to S^k and directly succeed $\mathsf{pos}(S^k)$ in the GESA [15, Lemma 3.1]. Given a prefix $S^t < S^k$, if a suffix of S^r, of length ℓ, is a prefix of S^t and $\ell > lcp(S^t, S^k)$, then such suffix of S^r is not a prefix of S^k [19, Lemma 2]. Furthermore, if two different suffixes of S^r are a prefix of S^k, the longest is closer to $\mathsf{pos}(S^k)$ in GESA.

The blocks are processed in order. For each block B^j, with $j = 1, 2, \ldots, m$, a local solution is found scanning B^j backwards and a global solution is obtained reusing the local solutions of the blocks processed previously.

For block B^j, GESA is scanned backwards, from $i = \mathsf{P}[j]$ to $\mathsf{P}[j-1]+1$. Suppose that $\mathsf{pos}(S^k) = \mathsf{P}[j]$ and $\mathsf{pos}(S^t) = \mathsf{P}[j-1]$. The algorithm uses m local lists and m global lists to track all overlaps seen so far. The value of $\ell = rmq(i, \mathsf{P}[j])$ is computed in $O(1)$ time as the minimum lcp value between the entries processed during the scanning of B^j. If the length of the current suffix is equal to $lcp(S^{\mathsf{STR}[i]}_{\mathsf{SA}'[i]}, S^k) = rmq(i, \mathsf{P}[j])$, then it is a prefix of S^k and its length is inserted at the end of its local list $L_{local}[\mathsf{STR}[i]]$. At the end, the longest overlaps in B^j are at the front of each local list, and ℓ is equal to $lcp(S^t, S^k)$.

The longest suffix of S^r that overlaps S^k may be positioned in a previous block $B^{j-1}, B^{j-2}, \ldots, B^1$ processed so far. The global lists store these overlaps. Each global list $L_{global}[r]$ is composed by the elements inserted in $L_{local}[r]$ and has to be updated as each block is processed. To do this, at the end of each local solution the algorithm removes the suffixes that have length larger than $lcp(S^t, S^k)$ from all local lists. These suffixes no longer overlap S^k or any the following complete strings that will be processed by the algorithm. Each local list $L_{local}[r]$ is prepended in the global list $L_{global}[r]$. Finally, the first element of each $L_{global}[r]$ corresponds to the length of the longest suffix of S^r that overlaps S^k. This value is inserted in $\mathsf{Ov}[k, r]$. Note that, to improve the memory access reference, the algorithm stores in $\mathsf{Ov}[k, r]$ the length of the longest suffix of S^r that overlaps S^k. If $L_{global}[r]$ is empty, there is no overlap of S^r with S^k.

The suffixes identical to S^k that directly succeed $\mathsf{pos}(S^k)$ are found scanning GESA forward from $i = \mathsf{P}[j] + 1$ to q, while $\mathsf{LCP}[q] = n_k$ [15, Lemma 3.1]. The length of these suffixes are inserted in Ov, possibly overwriting the results in $L_{global}[r]$, which is correct as such overlaps are larger.

4 Parallel Algorithm

In this section we show how to split the computation of all overlaps performed by Tustumi *et al.*'s algorithm to solve the APSP in parallel in a shared-memory multithreading environment.

At a glance, our algorithm is composed by three phases. First, all local solutions are computed scanning the blocks B^j concurrently. Then, the local lists are accessed in parallel to obtain the global solutions. Finally, the identical suffixes of all strings are identified scanning the GESA in parallel.

Algorithm: Suppose that for each block B^j, $pos(S^k) = \mathsf{P}[j]$ and $\mathsf{pos}(S^t) = \mathsf{P}[j-1]$. Algorithm 1 works as follows.

In Phase 1, the m blocks of the GESA are processed in parallel (Lines 1 to 12) to compute the local solutions of each string S^k. The algorithm scans each block B^j backwards (Lines 4 to 10), and whenever a suffix of S^r (Line 5) that overlaps S^k is found (Line 7), its length is inserted at the end of a local list, which is pointed by an entry in a two dimensional array (matrix) of lists, namely $L_{local}[r][j]$ (Line 8). Note that the list in $L_{local}[r][j]$ is ordered decreasingly by the overlap lengths due to the backward scan. An array of integers Min of length m is used to store in $\mathsf{Min}[j]$ the value of $rmq(\mathsf{P}[j-1], \mathsf{P}[j])$ (Line 11). At the end of Phase 1 (Line 12), all local solutions have been computed and are stored into the local lists, which will be used together with Min in Phase 2 to obtain the global solutions.

In Phase 2, the m arrays of local lists, that is, the m lines of matrix L_{local} that correspond to each S^r, are processed in parallel (Lines 13 to 23) to find the suffixes of S^r that overlap the other strings (Line 20). Let S^k be such overlapped string (Line 16). A global list for S^r is used to keep track of all valid overlaps in $L_{local}[r][j]$, according to $\mathsf{Min}[j]$, as blocks B^j are processed in the inner for-loop (Lines 15 to 22). L_{global} is initially empty (Line 14). For each B^j, with $j = 1, 2, \ldots, m$ (Lines 15 to 22), the algorithm updates L_{global}, first removing all suffixes larger than $\mathsf{Min}[j]$ (Lines 17 to 19), since these overlaps are no longer valid for S^k [19, Lemma 2]. Recall that $\mathsf{Min}[j] = rmq(\mathsf{P}[j-1], \mathsf{P}[j]) = lcp(S^t, S^k)$. In the sequel, $L_{local}[r][j]$ is prepended to L_{global} (Line 20) and the longest suffix of S^r that overlaps S^k will be the first element of L_{global} that is inserted at $\mathsf{Ov}[r, k]$ (Line 21).

In Phase 3, the suffixes identical to each strings S^k are found in parallel (Lines 24 to 33). For each block B^j and its corresponding string S^k (Line 25), all suffixes of S^r (Line 27) identical to S^k that appear directly after $\mathsf{pos}(S^k)$ in positions $i = \mathsf{P}[j] + 1$ to q are found (Lines 28 to 32). As in Tustumi *et al.*'s algorithm, the length of these suffixes are inserted in Ov (Line 29).

Theoretical Costs: In Phase 1, the parallel for loop in Line 1 is executed m/t times, where t is the number of threads. Then, Phase 1 is $O(\max_{1 \le j \le m} |B^j|)$. In Phase 2 the parallel for loop in Line 13 is executed m/t times, whereas Lines 15 to 22 are executed m^2/t times. The parallel for loop of Phase 3 is executed m/t times and each execution reads at most N elements of GESA. Thus our

Algorithm 1. Parallel APSP (p-apsp)

Data: GESA of the collection $\mathcal{S} = \{S^1, S^2 \ldots, S^m\}$
Result: result matrix Ov
// Local solutions
for $j \leftarrow 1$ *to* m **do in parallel**
  $k \leftarrow \mathsf{STR}[\mathsf{P}[j]]$;
  $\ell \leftarrow \infty$;
  for $i \leftarrow \mathsf{P}[j]$ *to* $\mathsf{P}[j-1]+1$ **do**
    $r \leftarrow \mathsf{STR}[i]$
    $\ell \leftarrow min(\ell, \mathsf{LCP}[i+1])$;
    if $|S^r_{\mathsf{SA}'[i]}| = \ell$ **then**
      $insert\_at\_end(L_{local}[r][j], \ell)$; // S^r overlaps S^k
    end
  end
  $\mathsf{Min}[k] \leftarrow \ell$;
end
// Global solutions
for $r \leftarrow 1$ *to* m **do in parallel**
  $L_{global} \leftarrow null$;
  for $j \leftarrow 1$ *to* m **do**
    $k \leftarrow \mathsf{STR}[\mathsf{P}[j]]$;
    while $first(L_{global}) > \mathsf{Min}[j]$ **do**
      $remove\_first(L_{global})$
    end
    $L_{global} \leftarrow insert\_at\_front(L_{local}[r][j])$;
    $\mathsf{Ov}[r,k] \leftarrow first(L_{global})$; // S^r overlaps S^k
  end
end
// Identical suffixes
for $j \leftarrow 1$ *to* m **do in parallel**
  $k \leftarrow \mathsf{STR}[\mathsf{P}[j]]$;
  $i \leftarrow \mathsf{P}[j]+1$;
  $r \leftarrow \mathsf{STR}[i]$;
  while $|S^r_{\mathsf{SA}'[i]}| = \mathsf{LCP}[i]$ *and* $i < N$ **do**
    $\mathsf{Ov}[r,k] \leftarrow \mathsf{LCP}[i]$; // $S^r_{\mathsf{SA}'[i]}$ is identical to S^k
    $i \leftarrow i+1$;
    $r \leftarrow \mathsf{STR}[i]$;
  end
end

algorithm runs in $O(N + m^2/t)$ time, which only occurs in bad cases (when the string lengths are very unbalanced). In realistic cases ($m \gg t$ and all strings with about the same size) the parallel time is close to $N/t + m^2/t$.

Overall, all threads insert at most N suffixes into the local lists. Then, the space complexity is given by the $O(N)$ space of the GESA, and by the $O(m^2+N)$ space of the matrix of local lists, where each list stores at most N overlaps.

Thus the space complexity of our parallel algorithm is $O(N + m^2)$, which is equal to the sequential algorithm by Tustumi *et al.*

5 Experiments

We implemented Algorithm 1 in C++ using OpenMP directives for the parallelization. We used the SDSL [4] version 2.0[1] to construct the GESA. Our source code is publicly available at https://github.com/felipelouza/p-apsp.

We have compared the performance of our parallel algorithm, called `p-apsp`, with the sequential optimal algorithm by Tustumi *et al.*[2], called `apsp`, and with the practical solution SOF [17][3], which has good time and space performance in multithreading environments. We used different number of threads $t = \{1, 2, 4, 8, 16, 32\}$ set by the directive `omp_set_num_threads()` for `p-apsp` and SOF. All programs were compiled with g++ (v. 4.9.2) with the same optimization flags.

The experiments were executed in a 64 bits Debian GNU/Linux 8 (kernel 3.16.0-4) system with an Intel Xeon Processor E5-2630 v3 20M Cache 2.40-GHz, with 384 GB of internal memory and a 13 TB SATA storage. We used the EST database from *C. elegans*[4]. The number of strings used in the experiments varied from 10.000 to 300.000. We used different minimum overlap length values $\tau = \{5, 10, 15, 20\}$ to limit the number of overlaps found by the algorithms.

We also tested the algorithms with 200.000 ESTs from *Citrus sinensis*[5] and obtained very close results, which are not shown.

Table 1 shows the number of overlaps found solving the APSP for the *C. elegans* dataset. Notice that as the value of τ increases the number of overlaps decreases. In particular, the number of overlaps for 300.000 strings when $\tau = 5$ is 23 times larger than when $\tau = 10$. We shall see that such variation impacts the performance of all algorithms, both in time and space.

Table 1. Number of overlaps found in the experiments with 100.000, 200.000 and 300.000 ESTs of the *C. elegans* varying τ.

τ	5	10	15	20
100.000	18,853,491	206,154	88,725	82,427
200.000	71,451,170	2,675,759	2,139,431	2,077,125
300.000	162,135,112	7,044,274	5,800,397	5,617,779

[1] *sdsl-lite* library is available at https://github.com/simongog/sdsl-lite.
[2] https://github.com/felipelouza/apsp.
[3] http://confluence.qu.edu.qa/download/attachments/9240580/Prefix.tgz.
[4] http://www.uni-ulm.de/in/theo/research/seqana.html.
[5] ftp://ftp.bioinfo.wsu.edu/www.citrusgenomedb.org/Citrus_sinensis/C.sinesis_unigene_v1.0/.

5.1 Running Time

Figure 2 shows the running time of each algorithm not accounting for the time to build the auxiliary data structures, that is, the GESA for apsp and p-apsp, and the compact prefix tree for SOF. The elapsed time was taken by the directive `omp_get_wtime()`.

SOF was the fastest algorithm in all experiments. However, p-apsp has shown a good performance when the minimum overlap length is small, a situation where the specialized strategy used by SOF is not so efficient. This result is coherent with the theoretical optimality of Tustumi *et al.*'s algorithm. Note that the parallel implementation has some overhead with $\tau = 5$ when comparing p-apsp with a single thread (p-apsp$_1$) to the sequential apsp. Note also that p-apsp and SOF improve their running time as the number of threads increases.

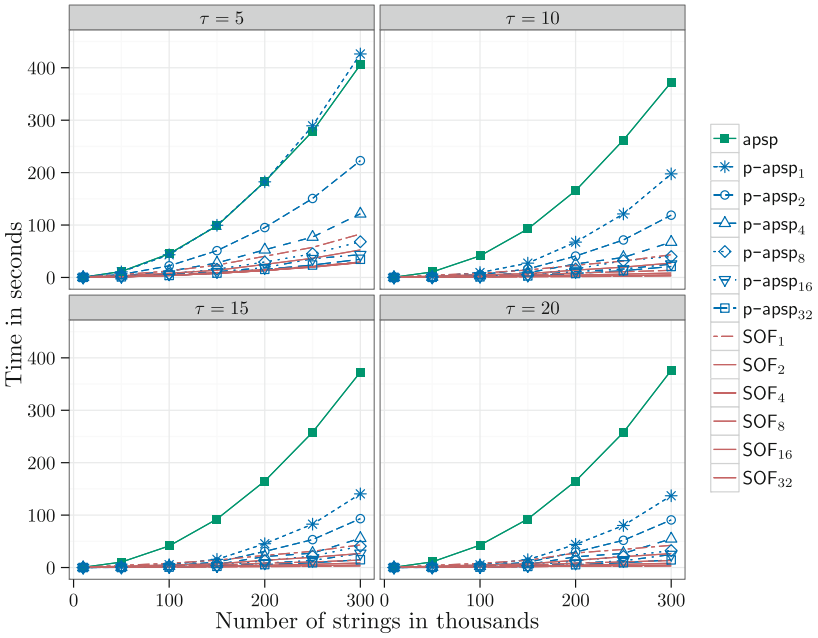

Fig. 2. Running time of p-apsp, apsp and SOF for varying values of τ.

Table 2 shows the running time and the speedup of each algorithm for 300.000 ESTs with $\tau = 5$. However, p-apsp achieved a much better speedup with the increasing number of threads, indicating that this parallel algorithm may be a practical solution for large instances of the APSP on strings coming from a general alphabet.

Table 2. Experiments with 300.000 ESTs of the *C. elegans* dataset with $\tau = 5$. The table shows the running time (in seconds) and the speedup of the parallel algorithms over its serial versions, when the numbers of threads is 1

n. threads	apsp	p-apsp		SOF	
	Time	Time	Speedup	Time	Speedup
1	397.17	463.33		**82.80**	
2		222.78	**1.91**	**52.49**	1.58
4		121.35	**3.51**	**29.50**	2.81
8		68.11	**6.26**	**28.30**	2.93
16		43.41	**9.82**	**28.64**	2.89
32		34.65	**12.31**	**23.62**	3.50

5.2 Peak Memory

The memory usage was measured by the malloc_count library[6]. We observed that the peak memory of p-apsp and SOF change slightly as the number of threads varies. For a clearer view we plot only the peak memory for p-apsp and SOF using 32 threads in Fig. 3.

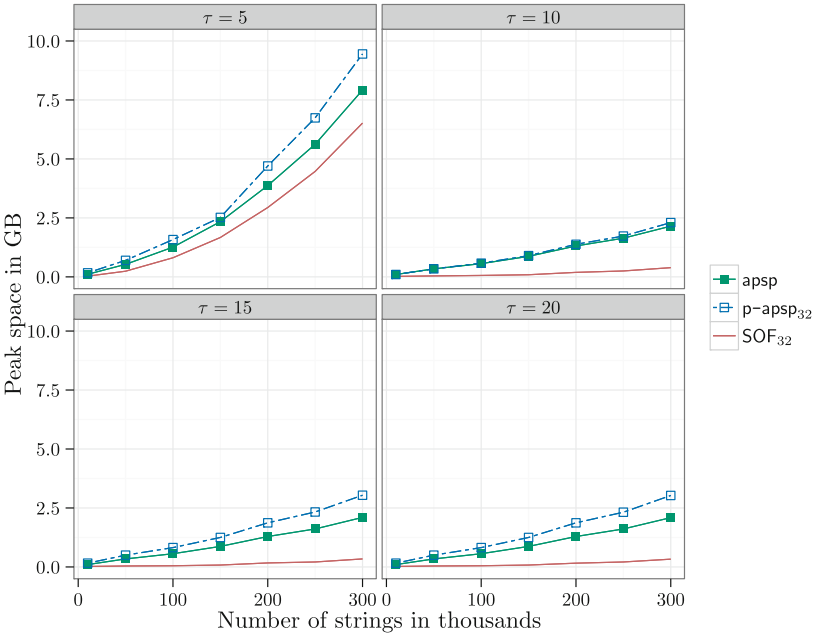

Fig. 3. Peak memory of p-apsp, apsp and SOF for varying values of τ

[6] *malloc_count* library is available at http://panthema.net/2013/malloc_count.

As expected, memory usage varies according to τ. SOF uses less memory in all experiments and p-apsp memory usage is very similar to the sequential version apsp. As demonstrated by the theoretical analysis the experiments confirm that practical memory usage differs only by a constant factor when comparing apsp and p-apsp.

6 Conclusion

We showed how to parallelize the optimal algorithm by Tustumi *et al.*[19] to solve the APSP. We separated the computation of the local solution followed by the global solution and by the identical suffixes searching into independent phases. We compared our parallel algorithms with SOF [17], a practical solution with the best parallel performance, as shown in previous work. Our experimental results showed that the parallel algorithm achieves a 12-fold speedup, and that it is competitive with practical algorithm, such as SOF, when the minimum overlap length is small and offers the ability to deal with larger alphabets.

As the algorithm by Tustumi *et al.*, our algorithm can be also improved to work in semi-external memory, since the GESA can be constructed in external memory [12] and its blocks can be accessed as necessary, reducing the peak memory. Our parallel implementation is general enough that it can be executed on a different architecture model, such as cloud distributed computing, possibly enabling the usage of hundreds of threads.

Acknowledgments. FAL acknowledges the financial support CAPES and CNPq (grant No. 162338/2015-5). GPT acknowledges the support of CNPq. The authors thank Prof. Nalvo Almeida for granting access to the machine used for the experiments.

References

1. Abouelhoda, M.I., Kurtz, S., Ohlebusch, E.: Replacing suffix trees with enhanced suffix arrays. J. Discrete Algorithms **2**(1), 53–86 (2004)
2. Dinh, H., Rajasekaran, S.: A memory-efficient data structure representing exact-match overlap graphs with application for next-generation DNA assembly. Bioinformatics **27**(14), 1901–1907 (2011)
3. El-Metwally, S., Hamza, T., Zakaria, M., Helmy, M.: Next-generation sequence assembly: four stages of data processing and computational challenges. PLoS Comput. Biol. **9**(12), e1003345 (2013)
4. Gog, S., Beller, T., Moffat, A., Petri, M.: From theory to practice: plug and play with succinct data structures. In: Gudmundsson, J., Katajainen, J. (eds.) SEA 2014. LNCS, vol. 8504, pp. 326–337. Springer, Heidelberg (2014)
5. Gonnella, G., Kurtz, S.: Readjoiner: a fast and memory efficient string graph-based sequence assembler. BMC Bioinform. **13**(1), 82 (2012)
6. Gonnet, G.H., Baeza-Yates, R.A., Snider, T.: New indices for text: pat trees and pat arrays. In: Information Retrieval, pp. 66–82. Prentice-Hall Inc, Upper Saddle River (1992)

7. Gusfield, D.: Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology. Cambridge University Press, New York (1997)
8. Gusfield, D., Landau, G.M., Schieber, B.: An efficient algorithm for the all pairs suffix-prefix problem. Inf. Process. Lett. **41**(4), 181–185 (1992)
9. Kalyanaraman, A., Aluru, S.: Handbook of computational molecular biology, chap. In: Expressed Sequence Tags: Clustering and applications. CRC Press, Boca Raton (2005)
10. Kasai, T., Lee, G.H., Arimura, H., Arikawa, S., Park, K.: Linear-time longest-common-prefix computation in suffix arrays and its applications. In: Amir, A., Landau, G.M. (eds.) CPM 2001. LNCS, vol. 2089, pp. 181–192. Springer, Heidelberg (2001)
11. Louza, F.A., Gog, S., Telles, G.P.: Induced suffix sorting for string collections. In: Proceeding DCC, pp. 43–52. IEEE, Snowbird (2016)
12. Louza, F.A., Telles, G.P., Ciferri, C.D.D.A.: External memory generalized suffix and LCP arrays construction. In: Fischer, J., Sanders, P. (eds.) CPM 2013. LNCS, vol. 7922, pp. 201–210. Springer, Heidelberg (2013)
13. Manber, U., Myers, E.W.: Suffix arrays: a new method for on-line string searches. SIAM J. Comput. **22**(5), 935–948 (1993)
14. Ohlebusch, E.: Bioinformatics Algorithms: Sequence Analysis, Genome Rearrangements, and Phylogenetic Reconstruction. Verlag, Oldenbusch (2013)
15. Ohlebusch, E., Gog, S.: Efficient algorithms for the all-pairs suffix-prefix problem and the all-pairs substring-prefix problem. Inf. Process. Lett. **110**(3), 123–128 (2010)
16. Puglisi, S.J., Smyth, W.F., Turpin, A.H.: A taxonomy of suffix array construction algorithms. ACM Comp. Surv. **39**(2), 1–31 (2007)
17. Rachid, M.H., Malluhi, Q.: A practical and scalable tool to find overlaps between sequences. BioMed Res. Int. **2015**, 1–12 (2015)
18. Simpson, J.T., Durbin, R.: Efficient construction of an assembly string graph using the FM-index. Bioinformatics **26**(12), i367–i373 (2010)
19. Tustumi, W.H., Gog, S., Telles, G.P., Louza, F.A.: An improved algorithm for the all-pairs suffix-prefix problem. J. Discrete Algorithms **47**, 34–43 (2016)
20. Weiner, P.: Linear pattern matching algorithms. In: Proceeding Annual Symposium on Switching and Automata Theory, pp. 1–11. IEEE Computer Society, Washington, DC (1973)